Cooperative Multi-Agent Deep Reinforcement Learning Methods for UAV-aided Mobile Edge Computing Networks

Mintae Kim, Hoon Lee, *Member, IEEE*, Sangwon Hwang, Mérouane Debbah, *Fellow, IEEE* and Inkyu Lee, *Fellow, IEEE*

arXiv:2407.03280v1 [cs.IT] 3 Jul 2024

***Abstract*—This paper presents a cooperative multi-agent deep reinforcement learning (MADRL) approach for unmmaned aerial vehicle (UAV)-aided mobile edge computing (MEC) networks. An UAV with computing capability can provide task offlaoding services to ground internet-of-things devices (IDs). With partial observation of the entire network state, the UAV and the IDs individually determine their MEC strategies, i.e., UAV trajectory, resource allocation, and task offloading policy. This requires joint optimization of decision-making process and coordination strategies among the UAV and the IDs. To address this difficulty, the proposed cooperative MADRL approach computes two types of action variables, namely message action and solution action, each of which is generated by dedicated actor neural networks (NNs). As a result, each agent can automatically encapsulate its coordination messages to enhance the MEC performance in the decentralized manner. The proposed actor structure is designed based on graph attention networks such that operations are possible regardless of the number of IDs. A scalable training algorithm is also proposed to train a group of NNs for arbitrary network configurations. Numerical results demonstrate the superiority of the proposed cooperative MADRL approach over conventional methods.**

***Index Terms*—Reinforcement learning, Graph attention network, UAV mobile edge computing.**

I. Introduction

Mobile edge computing (MEC) systems have been regarded as promising solutions to provide remote computation services for internet-of-things (IoT) networks with the aid of edge servers [1]–[9]. Edge servers mounted on unmanned aerial vehicles (UAVs) can further enhance the MEC performance by decreasing access distance between servers and IoT devices (IDs) [10]–[12]. The integration of IoT devices with the MEC enables more responsive and efficient handling of data, addressing latency-sensitive and bandwidth-intensive applications such as smart cities, healthcare monitoring, and industrial automation [13]–[15]. On the other hand, the mobility of UAVs incurs time-varying system dynamics, e.g., highly fluctuating propagation statistics. To tackle this difficulty, there have been studies to utilize the deep reinforcement learning (DRL) approaches [16]–[35].

Centralized DRL frameworks were developed for optimizing the trajectory of UAV servers and resource allocation strategies [16]–[25]. Deep Q-network (DQN) methods in [16]–[21] characterized these optimization variables as actions at an agent, i.e., UAV servers and IDs, which is realized by neural networks (NNs). Since the DQN is confined to discrete action spaces, it fails to identify continuous-valued UAV trajectory. Thus, suitable DRL schemes which can handle continuous action space were introduced such as deep deterministic policy gradient (DDPG), twin delayed DDPG (TD3), and proximal policy optimization (PPO) [22]–[25] based on an actor-critic architecture. The actor NN generates actions of UAV agents and ID agents, e.g., trajectory and resource allocation solutions, whereas the critic NN evaluates the effectiveness of the actor NN. By doing so, continuous-valued optimization variables can be successfully obtained for the UAV-based MEC systems.

A. Motivations

In practical UAV-aided MEC networks, the distributed operation is desirable so that observing states of environments and inferring actions for individual UAVs and IDs can be split across multiple UAVs and IDs. Existing centralized DRL methods [22]–[25] combine all UAVs and IDs as a single agent to handle states and actions. Such a single-agent deep reinforcement learning (SADRL) approach brings state collection and decision-making processes in the central manner, which are infeasible for supporting a massive number of IDs. Also, a sole actor-critic NN can only be trained to a certain MEC network with a fixed number of UAVs and IDs, and thus it cannot be straightforwardly applied to larger MEC systems.

One viable approach for distributed UAV-aided MEC systems is multi-agent DRL (MADRL) [26]–[35], which adopts multi-agent partially observable Markov decision processes (POMDP). Computing nodes at UAV servers and ground IDs are interpreted as agents which identify their actions based on partial observations of the overall MEC systems, e.g., channel state information, task volume, and locations. Various MADRL techniques such as the MAPPO [26]–[28], MADDPG [29]–[33] and MATD3 [34], [35] have shown their effectiveness for the decentralized management of UAVs and IDs.

M. Kim, S. Hwang, and I. Lee are with the School of Electrical Engineering, Korea University, Seoul 02841, Korea (e-mail: {wkd2749, tkddnjs3510, inkyu}@korea.ac.kr).

H. Lee is with the Department of Electrical Engineering and the AI graduate school, Ulsan National Institute of Science and Technology (UNIST), Ulsan, 44919, Korea (e-mail: hoonlee@unist.ac.kr).

M. Debbah is with Khalifa University of Science and Technology, P O Box 127788, Abu Dhabi, UAE (email: merouane.debbah@ku.ac.ae).

978-1-6654-3540-6/22 © 2022 IEEE

As the multi-agent POMDP formalism lacks full knowledge of the entire MEC system, a coordination strategy among agents should be employed so that they can estimate the overall state from partial observations. Two popular methods include implicit coordination through reward shaping [26]–[33] and exploit coordination through observation shaping [26]–[35]. The former trains all the agents jointly using carefully designed reward functions such that individual agents can implicitly infer the knowledge of other agents in the training. Since agents can cooperate in the training step via long-term statistics, trained actor NNs are not able to accommodate highly fluctuating environments. In contrast, the latter allows agents to share messages, which are normally given as subsets or manipulations of partial observations. These messages can be conveyed through reliable control channels. Such an explicit message exchange mechanism can adapt to immediate environment changes at the expense of increased communication overhead.

Both approaches have proven to succeed in various configurations of UAV-based MEC networks. However, these approaches generally require time-consuming trial-and-error validation processes to check the feasibility of all possible combinations of partial observations and rewards. This challenge may become prohibitive as the dimension of observation statistics increases. Moreover, man-made agent coordination policies are not guaranteed to achieve good performance since input features of actor NNs normally resort to manual optimization. For these reasons, it is necessary to develop a new cooperative MADRL framework that can autonomously determine interaction strategies among agents, in particular, communication messages by leveraging NNs.

B. Contributions and Organization

This paper investigates a cooperative MADRL for UAV-aided MEC networks where NNs at multiple agents determine their decision-making and coordination policies autonomously. We aim at minimizing the total energy consumption of ground IDs by offloading their computational tasks to a mobile edge server mounted on a UAV. This poses joint optimization of the trajectory and computing resource allocation of the UAV along with offloading decisions at IDs. These optimization variables need to be computed by the UAV and IDs individually. The considered problem is classified as a multi-agent POMDP where UAV and ID agents collaboratively identify their action variables only with partial observations. Compared to existing MADRL frameworks which require a handcraft design of UAV-ID coordination, the proposed scheme can handle highly fluctuating and heterogeneous network dynamics.

In this paper, we propose a novel cooperative multi-agent DDPG (C-MADDPG) framework where NNs at UAV and ID agents determine their policies by cooperating with other agents. Our system is first formulated as a cooperative multi-agent POMDP task which constructs two different actions for individual agents, namely, solution actions and message actions. The solution actions include optimization variables at agents, e.g., trajectory and resource allocation variables at the UAV agent and offloading variables at the ID agents. In addition, the message actions indicate communication messages to be exchanged among the UAV and ID agents. For effective information exchange, the proposed design establishes interactions in uplink (ID-to-UAV) and downlink (UAV-to-ID). Thus, along with the solution actions, the uplink message actions at IDs and the downlink message actions at the UAV are regarded as actions taken by actor NNs. This is a distinct feature of the proposed framework compared to conventional MADDPG approaches [36] where communication messages should be designed synthetically in advance and are fixed in all episodes of training steps.

In the proposed C-MADDPG, we build a solution actor NN and a message actor NN. In the inference stage, the UAV and ID agents calculate their solution actions by using the solution actor NN. Cooperative inference among these actor NNs establishes uplink-downlink coordination among the UAV and ID agents. To send encoded statistics of partial observations to the UAV agent, the ID agents utilize their message actor NNs that generate uplink message actions. These uplink message actions become an input to the message actor NN at the UAV agent, which creates the downlink message actions intended for individual ID agents. Once the coordination is completed, the UAV and ID agents calculate their solution actions by using the solution actor NNs. In this decision-making process, the message actions are leveraged as side inputs to the solution actor NNs so that the UAV and ID agents can collaboratively decide their optimization variables. Since the proposed cooperative inference does not need any centralized operations, we can construct decentralized optimization solutions for practical UAV-aided MEC networks.

In the proposed cooperative actor NN architecture, the UAV agent aggregates the uplink message actions sent by all ID agents. Therefore, the input dimension of actor NNs at the UAV agent, in particular, the message actor NN, scales with the number of IDs. For this reason, a naive NN structure whose input and output dimensions are fixed leads to poor generalization ability with respect to the ID population. To achieve the scalability, we exploit the concept of the graph attention network (GAT) [37], and adopt the parameter sharing technique where all ID agents utilize the identical actor NN. By doing so, the entire inference becomes independent of the number of IDs and can achieve the scalability.

The training process of the proposed C-MADDPG requests joint optimization of solution actor NNs, message actor NNs, and critic NNs. To this end, we adopt the centralized training decentralized execution (CTDE) strategy where all actor NNs are trained in an end-to-end manner under the supervision of the critic NN. The trained actor NNs are then deployed to the UAV and IDs for real-time decentralized inference. Consequently, the proposed parameter sharing policy can be implemented without additional communication overheads in the inference step. To further improve the scalablity of actor NNs, we develop a joint training process which leverages several episodes with the arbitrary number of ID agents. Also, we employ a random masking strategy that stochastically prunes input features of the critic NN. Numerical results validate the generalization ability of the proposed scheme and demonstrate the effectiveness of the proposed C-MADDPG

framework over existing approaches.

The contributions of this paper are summarized as follows:

- We propose a novel C-MADDPG framework which establishes self-organizing coordination strategies among UAV and ID agents. Compared to existing MADRL methods [26]–[35] which design agent coordination messages manually, the proposed approach exploits message actor NNs to allow autonomous RL operations. This framework generates task-oriented agent interaction protocols that are optimized to enhance the expected reward function. Consequently, the proposed method does not require a handcrafted design of observations and rewards.
- In practical UAV-aided MEC networks, the number of IDs may vary from time to time. This requests actor NNs whose inference calculations can be performed independent of the ID populations. To this end, we develop scalable actor NNs based on the parameter sharing strategy where all IDs leverage the identical NNs. However, a straightforward extension of the parameter sharing entails indistinguishable messages at all IDs. To address this difficulty, the GAT mechanism is employed which evaluates the importance of individual IDs and the resulting actor NNs successfully achieve the scalability to the number of IDs.
- To further enhance the scalability, the training mechanism of the proposed C-MADDPG should be carefully designed such that shared actor NNs can observe various MEC configurations with arbitrary ID populations. Such randomized samples can be generated by using masking operations. We randomly prune IDs in the training step so that the actor NNs can be optimized over different MEC networks. This strategy helps the NNs to learn an efficient decentralized decision-making policy for arbitrary given ID populations.

The rest of the paper is organized as follows: Section II offers an overview of recent works on MADRL-based UAV MEC systems. In Section III, we describe a system model and formulate an optimization problem. Section IV provides a cooperative POMDP formulation. The proposed actor structure and its cooperative inference are presented in Section V. In Section VI, a joint training policy is introduced, and the performance evaluations are shown in Section VII. Finally, the paper is terminated with concluding remarks in Section VIII.

II. Related Works

Centralized single-agent DRL (SADRL) has been developed to tackle various optimization problems in UAV-aided MEC networks [16]–[25]. In [16], the utility maximization problem of a UAV server was studied based on the DQN framework. The energy consumption was minimized by scheduling the offloading and the position of the UAV using DQN [17]. To obtain continuous-valued UAV trajectories, the DDPG and PPO were taken into account [22]–[25]. The DDPG method [22] determined the UAV trajectory to minimize the energy consumption of IDs. The minimization of task completion latency and energy consumption was considered in [25].

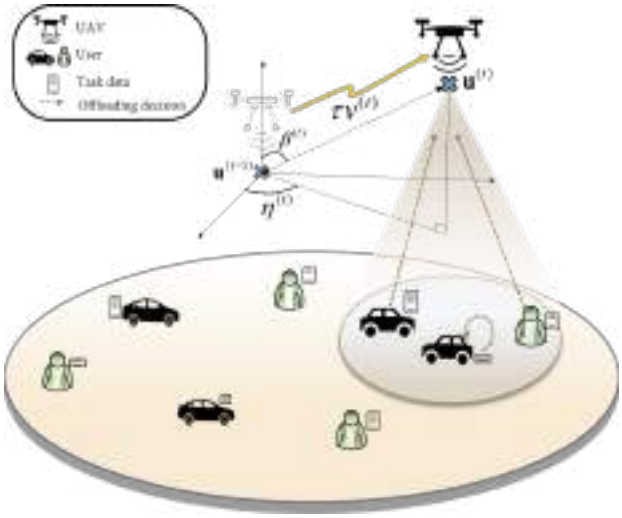

Fig. 1: UAV-assisted MEC system model

To reduce the complexity of the SADRL, there has been a recent paradigm shift towards the MADRL [26]–[35]. These methods offer an effective mechanism for dealing with cooperative or competitive interactions within intricate environments. In [26], the deployment of MEC-powered UAVs was optimized for sub-THz communication. A modified MAPPO was proposed in [27] to handle the energy consumption minimization problem effectively. A joint optimization problem of precoder, trajectory, and ID association was solved in an integrated sensing and communication network [28]. A fairness maximization problem was examined in [30] by optimizing trajectory and offloading decisions. In [31], total delay and energy consumption were minimized by adopting the stochastic game. The ratio of the transmission rate to the energy consumption of a UAV was optimized in [32] by combining the game theory and MADDPG. The authors in [33] maximized the number of offloaded tasks while meeting heterogenous quality-of-service requirements. The multi-UAV multi-clouds task offloading problems were addressed in [34].

These MADRL approaches [26]–[35] generally require exhaustive search processes for identifying efficient rewards and observations heuristically. Weighted sum reward functions were considered in [27], [28], [32] where the optimized weights should be found numerically. The work in [31] designed the reward as the difference between local and edge computing cost, whereas [33] employed the smoothen objective function as the reward. Also, in [34] and [35], the UAV agents exchange their current locations, and these communication messages contribute to partial observation inputs for actor NNs. These man-made agent coordination polices are computationally inefficient and even become infeasible for practical UAV MEC systems with a number of heterogeneous observations and actions.

III. Network Model

As illustrated in Fig. 1, we consider a UAV-assisted MEC system where the UAV server flies over the network area to

offer computation offloading services for N mobile IDs at the ground. A time-slotted MEC protocol is adopted where the system block is divided into T time slots. Let $\mathcal{N} \triangleq \{1, \cdots, N\}$ be the index set of IDs. ID j ($\forall j \in \mathcal{N}$) desires to handle its computational task of size I_j bits within one system block consisting of T time slots.

A. Mobility Model

Let $\mathbf{q}_j^{(t)} = (q_{x,j}^{(t)}, q_{y,j}^{(t)}, 0) \in \mathbb{R}^3$ be the 3D Cartesian coordinate of ID j at time slot t ($t = 1, \cdots, T$). Mobile IDs change their positions time to time according to predefined missions. The randomness in ID positions can be modeled by the Gauss-Markov process [38]. At time slot t, the location of ID j is written by

$$q_{x,j}^{(t)} = q_{x,j}^{(t-1)} + \tau v_j^{(t)} \cos o_j^{(t)}, \tag{1a}$$

$$q_{y,j}^{(t)} = q_{y,j}^{(t-1)} + \tau v_j^{(t)} \sin o_j^{(t)}, \tag{1b}$$

where τ represents the duration of a time slot and the speed $v_j^{(t)}$ and $o_j^{(t)}$ indicate speed and moving direction, respectively. Here, $v_j^{(t)}$ and $o_j^{(t)}$ are updated as

$$v_j^{(t)} = \kappa_v v_j^{(t-1)} + (1-\kappa_v)\bar{v}_j + \sqrt{1-\kappa_v^2}\Phi_v, \tag{2a}$$

$$o_j^{(t)} = \kappa_o o_j^{(t-1)} + (1-\kappa_o)\bar{o}_j + \sqrt{1-\kappa_o^2}\Phi_o, \tag{2b}$$

where $\kappa_v \in [0,1]$ and $\kappa_o \in [0,1]$ stand for the memory factors and $\bar{v}_j$ and $\bar{o}_j$ are the average speed and direction of ID j, respectively. The independent Gaussian random variables $\Phi_v \sim N(0, \varsigma_v^2)$ and $\Phi_o \sim N(0, \varsigma_o^2)$ characterize the randomness of the ID mobility. In the meantime, the UAV trajectory is optimized to enhance the MEC performance. Let us define $\beta^{(t)} \in [0, 2\pi]$ and $\eta^{(t)} \in [0, \pi]$ respectively as the azimuth angle and the elevation angle of the UAV at time slot t. Then, the moving direction of the UAV $\mathbf{\Delta}^{(t)} \in \mathbb{R}^3$ can be expressed as

$$\mathbf{\Delta}^{(t)} = (\sin\beta^{(t)}\cos\eta^{(t)}, \sin\beta^{(t)}\sin\eta^{(t)}, \cos\beta^{(t)}). \tag{3}$$

As a result, the 3D location vector of the UAV is obtained as

$$\mathbf{u}^{(t)} = \mathbf{u}^{(t-1)} + \tau v^{(t)} \mathbf{\Delta}^{(t)}, \tag{4}$$

where $v^{(t)} \in [0, v_{\max}]$ equals the UAV velocity with $v_{\max}$ being the maximum speed constraint.

B. Channel Model

We define $P_j^{(t)}$ as the line of sight (LoS) probability given by [39]

$$P_j^{(t)} = \frac{1}{1 + K_1 \exp\left(-K_2[\nu_j^{(t)} - K_1]\right)}, \tag{5}$$

where K_1 and K_2 are constants on the propagation environment and $\nu_j^{(t)}$ equals the elevation angle between the UAV and ID j. According to the air-to-ground propagation model [40], [41], the large-scale channel gain between the UAV and ID j can be written by

$$h_j^{(t)} = \frac{\|\mathbf{u}^{(t)} - \mathbf{q}_j^{(t)}\|^{-\alpha}}{\rho_0(P_j^{(t)}\chi_{\text{LoS}} + (1-P_j^{(t)})\chi_{\text{NLoS}})}, \tag{6}$$

where $\mathbf{q}_j^{(t)}$ is the 3D location vector of ID j, α represents the path loss exponent, ρ_0 indicates the reference path loss, and χ_{LoS} and χ_{NLoS} ($\chi_{\text{NLoS}} > \chi_{\text{LoS}} > 1$) respectively account for the path loss of the LoS and non-LoS cases.

We employ the time division duplexing protocol where uplink and downlink communication are realized over the reciprocal channel. The uplink and downlink rates are respectively expressed by

$$R_{u,j}^{(t)} = \frac{B}{N}\log_2\left(1 + \frac{N p_u h_j^{(t)}}{B N_0}\right), \tag{7}$$

$$R_{d,j}^{(t)} = \frac{B}{N}\log_2\left(1 + \frac{N p_d h_j^{(t)}}{B N_0}\right), \tag{8}$$

where p_u and p_d equal the uplink and downlink transmit power at the IDs and the UAV, respectively, B denotes the total bandwidth and N_0 stands for the noise power.

C. Offloading Process

ID j splits its task into T subtasks each having I_j/T bits. Each subtask is subject to be completed within one time slot of duration τ. At the beginning of each time slot, the IDs determine their task offloading policies based on the partial offloading protocol. ID j offloads $\lambda_j^{(t)} \in [0,1]$ portion of I_j/T bits to the UAV server, whereas the remaining part $1-\lambda_j^{(t)}$ is processed locally. The energy consumption $E_{l_j}^{(t)}$ required for the local processing of $(1-\lambda_j^{(t)})I_j/T$ bits is written as [42]

$$E_{l_j}^{(t)} = \vartheta \frac{(C(1-\lambda_j^{(t)})I_j)^3}{\tau^2 T^3}, \tag{9}$$

where the constants ϑ and C account for the hardware efficiency and the computational complexity, respectively.

To offload $\lambda_j^{(t)}\frac{I_j}{T}$ bits to the UAV, the communication energy $E_{o_j}^{(t)}$ of ID j is given by

$$E_{o_j}^{(t)} = \frac{p_u \lambda_j^{(t)} I_j}{R_{u,j}^{(t)} T}. \tag{10}$$

The computation capacity of the UAV is limited by the maximum CPU frequency $f_{\max}$. For parallel computations, a virtual machine (VM) [43] with the CPU frequency $f_j^{(t)}$ is dedicated to processing the task offloaded from ID j. This incurs the sum CPU frequency constraint as

$$\sum_{j=1}^{N} f_j^{(t)} \le f_{\max}, \forall t. \tag{11}$$

The latency of the offloading procedure at ID j comprises delays in the uplink task offloading from ID j to the UAV, task computation at the UAV, and downlink transmission from the UAV to ID j as

$$L_{o_j}^{(t)} = \frac{\lambda_j^{(t)} I_j}{R_{u,j}^{(t)} T} + \frac{C\lambda_j^{(t)} I_j}{f_j^{(t)} T} + \frac{\delta\lambda_j^{(t)} I_j}{R_{d,j}^{(t)} T}, \tag{12}$$

where the first term represents the uplink transmission delay of the offloaded task of size $\lambda_j^{(t)} I_j$ with $R_{u,j}^{(t)}$ bits/sec, and the second term indicates the computation latency with the CPU

frequency $f_j^{(t)}$ cycles/sec, and the third term quantifies the downlink transmission delay for broadcasting the task of size $\delta\lambda_j^{(t)} I_j$ with $R_{d,j}^{(t)}$ bits/sec with the constant δ being the ratio of output to input task sizes. Finally, the latency constraint is imposed as

$$L_{o_j}^{(t)} \leq \tau, \forall j, t. \tag{13}$$

D. Problem Description

We aim at minimizing the total energy consumption of all IDs through the joint optimization of the UAV trajectory $\mathbf{U} = \{v^{(t)}, \eta^{(t)}, \beta^{(t)}, \forall t\}$, computing resource allocation $\mathbf{F} = \{f_j^{(t)}, \forall j, t\}$, and offloading ratio $\mathbf{\Lambda} = \{\lambda_j^{(t)}, \forall j, t\}$. The total energy minimization problem can be formulated as

$$\min_{\mathbf{U},\mathbf{F},\mathbf{\Lambda}} \quad \frac{1}{T}\sum_{t=1}^{T}\sum_{j=1}^{N} E_{l_j}^{(t)} + E_{o_j}^{(t)}$$

$$\text{s.t.} \quad v^{(t)} \in [0, v_{\max}],\ \eta^{(t)} \in [0, 2\pi),\ \beta^{(t)} \in [0, \pi], \tag{14a}$$

$$f_j^{(t)} \geq 0, \tag{14b}$$

$$\lambda_j^{(t)} \in [0, 1], \tag{14c}$$

$$(11) \text{ and } (13). \tag{14d}$$

The above problem is a nonconvex problem due to the latency constraint and the objective function. Existing MADRL methods [26]–[35] resort to computationally demanding exhaustive search processes for designing agent interaction mechanisms. To overcome this difficulty, we propose a novel cooperative MADRL framework that identifies optimization variables and coordination strategies autonomously by using NNs.

IV. Cooperative Multi-Agent POMDP Formulation

We introduce a C-MADDPG scheme for addressing $(\mathbf{P})$ where the UAV and IDs are realized as individual agents taking their decision variables. Separate UAV and IDs independently interact with the environment, i.e., the MEC network. This leads to a cooperative POMDP formulation where each agent can only access to partial knowledge on the current environment. In what follows, we transform $(\mathbf{P})$ into the multi-agent POMDP task consisting of states, observations, actions, and rewards.

1) Observations and States

We denote the UAV as agent 0 and ID j as agent j. Let $\tilde{\mathcal{N}} \triangleq \{0\} \bigcup \mathcal{N}$ be the set of all agents. The partial observation $o_j^{(t)}$ of agent $j \in \tilde{\mathcal{N}}$ at time slot t consists of information about the entire MEC network that can be observed by agent j. For the UAV agent, the observation $o_0^{(t)}$ is set to its previous location $\mathbf{u}^{(t-1)}$ as

$$o_0^{(t)} = \mathbf{u}^{(t-1)}. \tag{15}$$

In contrast, ID agent j forms its observation $o_j^{(t)}$ as

$$o_j^{(t)} = \left\{ \mathbf{q}_j^{(t-1)}, (1-\lambda_j^{(t-1)})\frac{I_j}{T}, \lambda_j^{(t-1)}\frac{I_j}{T}, I_j, R_{u,j}^{(t-1)} \right\}. \tag{16}$$

As a result, the state of the MEC network $s^{(t)}$ collects all observations as

$$s^{(t)} \triangleq \{o_j^{(t)} : \forall j \in \tilde{\mathcal{N}}\}. \tag{17}$$

2) Solution Actions

The solution action $x_j^{(t)}$ contains a set of optimization variables identified by agent j. As discussed, the solution action $x_0^{(t)}$ of the UAV agent receives the trajectory variables as

$$x_0^{(t)} = \{\mathbf{v}^{(t)}, \mathbf{f}^{(t)}\}, \tag{18}$$

where $\mathbf{v}^{(t)} = \{v^{(t)}, \eta^{(t)}, \beta^{(t)}\}$ is the trajectory variable and $\mathbf{f}^{(t)} \triangleq \{f_j^{(t)} : \forall j \in \mathcal{N}\}$ stands for a collection of the CPU frequencies of all IDs. Also, ID agent j obtains its own offloading decision variable $\lambda_j^{(t)}$. Thus, the solution $x_j^{(t)}$ of ID agent j becomes

$$x_j^{(t)} = \lambda_j^{(t)}. \tag{19}$$

3) Reward

The reward function $r^{(t)}$ evaluates the performance of the MEC network at time slot t. Since our aim is to minimize the energy consumption of all IDs, the reward $r^{(t)}$ is set to

$$r^{(t)} = -\sum_{j=1}^{N}(E_{l_j}^{(t)} + E_{o_j}^{(t)}). \tag{20}$$

4) Message Actions

The considered POMDP formulation can be addressed by the conventional MADDPG framework [36]. In this approach, agent j ($\forall j \in \tilde{\mathcal{N}}$) is equipped with its own actor NN, which produces the solution action $x_j^{(t)}$ from the partial observation $o_j^{(t)}$. The major drawback of the conventional MADDPG comes from the limited agent interaction. Since the solution actions of all agents are highly coupled in the UAV-aided MEC networks, coordination among the UAV and IDs is essential for identifying the optimal solution to **(P)**. Nevertheless, the actor DNN only accepts the partial observation $o_j^{(t)}$ as an input, and thus the resulting solution action $x_j^{(t)}$ is determined without knowing the observations of other agents.

To cope with this issue, along with the decision processes of the solution actions, we develop a coordination policy among the UAV and ID agents, which can be realized by additional message actions $m_j^{(t)}$ ($\forall j \in \tilde{\mathcal{N}}$). The message actions should be designed to encapsulate sufficient statistics of agent j needed for individual decision-making processes at others. Messages of ID agents are shared with the UAV through uplink control channels. Similarly, the UAV multicasts its message action $m_0^{(t)}$ to all IDs via downlink control channels. As will be explained, the message actions are determined using additional actor NNs, and the resulting message actors are leveraged as side information to decide the solution actions. Thus, the overall action of agent j consists of both the solution action and message action as

$$a_j^{(t)} = \{x_j^{(t)}, m_j^{(t)}\}. \tag{21}$$

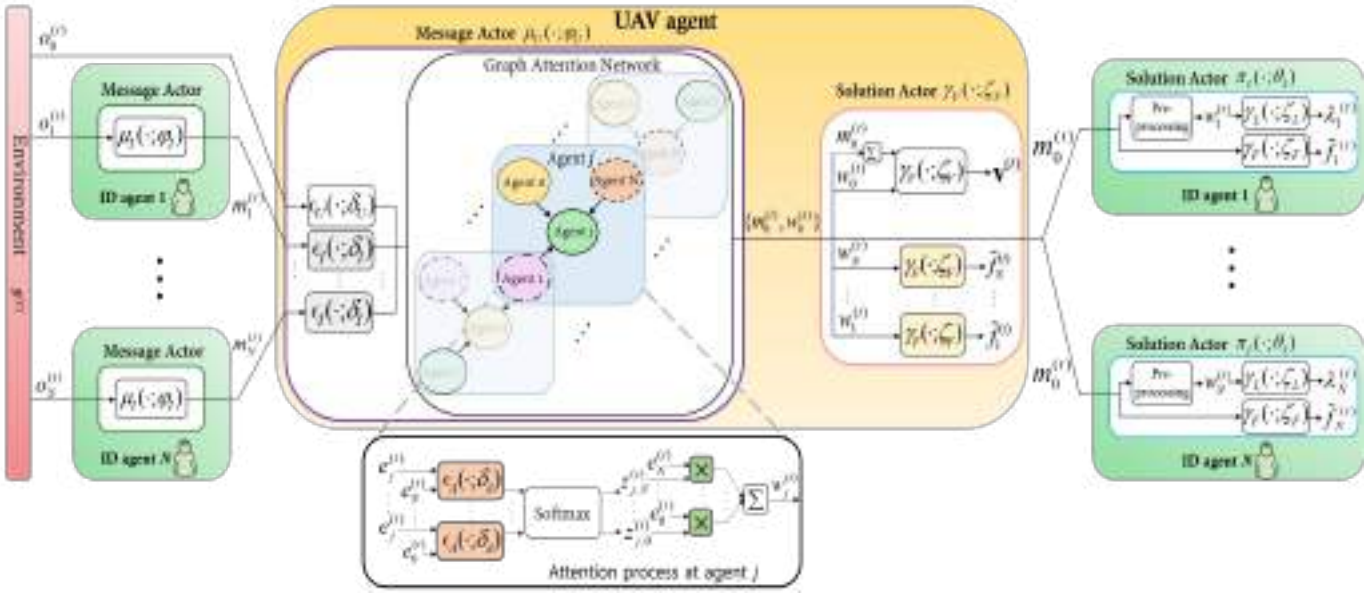

Fig. 2: Proposed cooperative actor architecture

V. Cooperative Actor Design

The cooperative multi-agent POMDP formulation presented in Section IV readily establishes the C-MADDPG framework to take the solution actions $x_j^{(t)}$ ($\forall j \in \tilde{\mathcal{N}}$) using decentralized coordination among the UAV and ID agents. In the C-MADDPG method, this can be achieved by employing actor NNs at individual agents. In order to determine the message actions $m_j^{(t)}$ ($\forall j \in \tilde{\mathcal{N}}$), we employ additional actor NNs that produce message action variables. As illustrated in Fig. 2, the proposed architecture deploys two types of actors: *message actor* and *solution actor*. The message actors determine agent coordination, whereas the solution actors compute appropriate solution variables based on the received message actions. Such a cooperative actor architecture leads to the joint optimization of two different actions in a goal-oriented manner for maximizing the expected reward value.

The proposed actor design invokes a challenging issue on the scalability with respect to the network size, in particular, the number of IDs N. The joint optimization of message actors and solution actors poses the exploding population of trainable parameters that is proportional to $2N$. Such dedicated actor NNs lack the generalization ability to an arbitrary N. Also, the sizes of the state $s^{(t)}$ and message actions $\{m_j^{(t)} : \forall j \in \mathcal{N}\}$ grow with N, which increases the model complexity for critic and actor NNs to handle high-dimensional states and actions. To address these difficulties, in this section, we develop a cooperative and scalable actor structure whose message-generating and solution-optimizing computations become independent of the number of IDs. In what follows, we discuss the inference steps of message actors and solution actors.

A. Message Actors at ID agents

ID agent j ($\forall j \in \mathcal{N}$) first obtains its message action $m_j^{(t)}$ by the message actor NN $\mu_I(\cdot;\varphi_I)$ with trainable parameter φ_I based on its partial observation $o_j^{(t)}$. The message action $m_j^{(t)}$ of ID agent j ($\forall j \in \mathcal{N}$) is then expressed as

$$m_j^{(t)} = \mu_I(o_j^{(t)};\varphi_I), \tag{22}$$

where the identical message actor $\mu_I(\cdot;\varphi_I)$ is employed for all ID agents. Such a parameter sharing policy leads to a scalable structure so that a sole message actor can be universally applied to an arbitrary ID population.

A set of ID messages $\{m_j^{(t)} : \forall j \in \mathcal{N}\}$ are conveyed to the UAV agent through orthogonal uplink control channels. Without loss of the generality, each ID is assumed to be assigned by M frequency resource blocks (RBs) to transmit its message action. To accommodate such a resource constraint, we design $m_j^{(t)}$ as an M-dimensional vector where the transmission of each element occupies one RB. As a result, total NM RBs are needed for the uplink coordination.

B. Message Actor at UAV agent

After receiving the messages $\mathbf{m}_I^{(t)} \triangleq \{m_j^{(t)} : \forall j \in \mathcal{N}\}$, the UAV agent computes its message action $m_0^{(t)}$ to be multicast to all IDs through the downlink control channels. This UAV message action encodes the knowledge required for the decision processes of the solution action $x_j^{(t)}$ at ID agent $j \in \mathcal{N}$. To this end, the UAV agent combines its observation $o_0^{(t)}$ and the group of ID message actions $\mathbf{m}_I^{(t)}$. By doing so, the UAV successfully propagates partitioned information of individual IDs to the entire MEC network. Similar to the ID message actions, the UAV message actor $\mu_U(\cdot;\varphi_U)$ with trainable parameter φ_U is adopted to produce the UAV message action $m_0^{(t)}$ as

$$m_0^{(t)} = \mu_U(o_0^{(t)}, \mathbf{m}_I^{(t)};\varphi_U). \tag{23}$$

The dimension of the ID message actions $\mathbf{m}_I^{(t)} \in \mathbb{R}^{NM}$ scales with the number of IDs N. For this reason, a naive

NN architecture, in particular, fully-connected layers, fails to achieve the scalability with respect to N.

To overcome this issue, we develop a scalable UAV message actor $\mu_U(\cdot;\varphi_U)$ based on the GAT [37]. This framework modifies node interaction policies of the graph neural network (GNN) [44] such that each node can measure the importance of its neighbors, which is referred to as an attention score. As a result, the generalization capability can be fairly improved without sacrificing the scalability to the node population. Thus, the design goal of the message actor (23) is to aggregate the observation of the UAV agent $o_0^{(t)}$ and $\mathbf{m}_I^{(t)}$ based on the importance of individual IDs.

In general, the GAT facilitates multiple layers to extract useful features of input data. To realize such a layered GAT architecture, several communication rounds among the UAV and IDs are necessary to share the results of each GAT iteration. To avoid this issue, we modify conventional multi-iteration GAT architectures to a single-iteration GAT to leverage sole uplink-downlink coordination. Also, since the DRL involves temporal connections of actor NNs in consecutive time slots, our single-iteration GAT design becomes more powerful in the UAV-aided MEC systems.

A key enabler of the proposed GAT approach is to allow the UAV agent to have NN modules for computing the attention scores of all ID agents. By doing so, we can straightforwardly implement the GAT mechanism without propagating latent vectors of hidden layers multiple times. The UAV agent first extracts a hidden feature of agent $j \in \tilde{\mathcal{N}}$, denoted by $\mathbf{e}_j^{(t)}$ of length E, as

$$\mathbf{e}_j^{(t)} = \begin{cases} \epsilon_U(o_0^{(t)};\delta_U) \text{ for } & j=0, \\ \epsilon_I(m_j^{(t)};\delta_I) \text{ for } & j \in \mathcal{N}, \end{cases} \tag{24}$$

where $\epsilon_U(\cdot;\delta_U)$ and $\epsilon_I(\cdot;\delta_I)$ indicate the feature extractor NNs of the UAV agent and ID agents, respectively, which are responsible for generating $\mathbf{e}_j^{(t)}$ utilized for the computation of the attention scores. A group of feature vectors $\{\mathbf{e}_j^{(t)} : \forall j \in \tilde{\mathcal{N}}\}$ is adopted as an input to the GAT operation. A scalar attention score $z_{j,k}^{(t)} \in [0,1]$ about agent k measured by agent j is calculated as

$$z_{j,k}^{(t)} = \frac{\exp(\epsilon_A(\mathbf{e}_j^{(t)}, \mathbf{e}_k^{(t)};\delta_A))}{\sum_{i\in\tilde{\mathcal{N}}} \exp(\epsilon_A(\mathbf{e}_j^{(t)}, \mathbf{e}_i^{(t)};\delta_A))}, \tag{25}$$

where $\epsilon_A(\cdot;\delta_A)$ stands for the attention NN that evaluates the affinity of two different agents. The attention score $z_{j,k}^{(t)}$ interprets the importance of agent k for the decision-making process at agent j. Finally, the output of the GAT for agent j becomes the weighted average of the feature vectors $\{\mathbf{e}_j^{(t)} : \forall j \in \tilde{\mathcal{N}}\}$ with coefficients $\{z_{j,k}^{(t)} : \forall j \in \tilde{\mathcal{N}}\}$ as

$$\mathbf{w}_j^{(t)} = \sum_{k\in\tilde{\mathcal{N}}} z_{j,k}^{(t)} \mathbf{e}_k^{(t)}. \tag{26}$$

Notice that in conventional GAT, each agent has its dedicated feature extractor NN. Thus, to obtain the attention score $z_{j,k}^{(t)}$ in (25), ID agent j should know all the feature vectors $\mathbf{e}_k^{(t)}$ ($\forall k \in \tilde{\mathcal{N}}$) which is not viable with sole uplink cooperation from IDs to the UAV. This can be addressed by allowing the UAV agent to reuse the feature extractor NNs $\epsilon_I(\cdot;\delta_I)$ for all IDs.

Thanks to the GAT mechanism, $\mathbf{w}_j^{(t)}$ encodes sufficient statistics required at agent j to take its solution actions. Thus, they can be utilized as an input of solution actor NNs. As will be discussed, $\mathbf{w}_0^{(t)}$ is leveraged internally to find the solution action of the UAV agent $x_0^{(t)}$. In contrast, the remaining vectors $\mathbf{w}_j^{(t)}$ ($\forall j \in \mathcal{N}$) need to be sent to the associated ID agents. To this end, the UAV message action $m_0^{(t)}$ is designed as

$$m_0^{(t)} = \{\mathbf{w}_j^{(t)} : \forall j \in \mathcal{N}\}. \tag{27}$$

The UAV multicasts $m_0^{(t)} \in \mathbb{R}^{NE}$ to the IDs, which occupies NE RBs for the downlink coordination.

C. Solution Actor at UAV agent

The UAV message $m_0^{(t)}$ encapsulates the network state $s^{(t)}$, i.e., the set of all observations of the UAV and ID agents. Therefore, it is sufficient for all agents to determine their solution actions by leveraging $m_0^{(t)}$ only. Let $\pi_U(\cdot;\theta_U)$ be the solution actor NN at the UAV agent with the learnable parameter θ_U. The UAV solution action $x_0^{(t)}$ is obtained as

$$x_0^{(t)} = \pi_U(m_0^{(t)};\theta_U). \tag{28}$$

As shown in (18), the solution action of the UAV agent $x_0^{(t)}$ contains two types of optimization variables, i.e., the trajectory $\mathbf{v}^{(t)}$ and computing resource allocation $\mathbf{f}^{(t)}$. To yield such heterogeneous actions, the solution actor $\pi_U(\cdot;\theta_U)$ comprises two component NNs $\gamma_V(\cdot;\zeta_V)$ and $\gamma_F(\cdot;\zeta_F)$ for calculating $\mathbf{v}^{(t)}$ and $\mathbf{f}^{(t)}$, respectively. Then, the trainable parameter set of the solution actor NN of the UAV becomes $\theta_U = \{\zeta_V, \zeta_F\}$.

The trajectory variable is computed as

$$\mathbf{v}^{(t)} = \gamma_V\left(\mathbf{w}_0^{(t)}, \sum_{j\in\mathcal{N}} \mathbf{w}_j^{(t)};\zeta_V\right), \tag{29}$$

where an input is a concatenation of the UAV information vector $\mathbf{w}_0^{(t)} \in \mathbb{R}^E$ from the GAT (25) and the sum of the ID information vectors $\sum_{j\in\mathcal{N}} \mathbf{w}_j^{(t)} \in \mathbb{R}^E$. Since the dimension of these input vectors is independent of N, (29) preserves the scalability with respect to the ID population. The distinct input $\mathbf{w}_0^{(t)}$ helps the NN $\gamma_V(\cdot;\zeta_V)$ distinguish the UAV information vector with those of the IDs $\sum_{j\in\mathcal{N}} \mathbf{w}_j$. Consequently, we can successfully produce the UAV-specific trajectory action $\mathbf{v}^{(t)}$ based on the aggregated ID information $\sum_{j\in\mathcal{N}} \mathbf{w}_j$.

Next, to determine the computing resource allocation $\mathbf{f}^{(t)}$, a sole NN produces each CPU frequency variable $f_j^{(t)}$ based on the associated ID agent information vector $\mathbf{w}_j^{(t)}$. To this end, we first calculate an intermediate value $\tilde{f}_j^{(t)}$ as

$$\tilde{f}_j^{(t)} = \gamma_F(\mathbf{w}_j^{(t)};\zeta_F), \tag{30}$$

where the output activation of $\gamma_F(\cdot;\zeta_F)$ is set to the rectified linear unit (ReLU) to yield a nonnegative number $\tilde{f}_j^{(t)}$. Upon obtaining all $\tilde{f}_j^{(t)}$, the CPU cycle action $f_j^{(t)}$ is retrieved as

$$f_j^{(t)} = \frac{\tilde{f}_j^{(t)}}{\sum_{k\in\mathcal{N}} \tilde{f}_k^{(t)}} f_{\max}, \tag{31}$$

which forces $\mathbf{f}^{(t)}$ to be feasible for the computing resource constraint (11).

D. Solution Actor at ID agent

From the message action $m_0^{(t)}$ from the UAV, ID agent j first recovers its corresponding information vector $\mathbf{w}_j^{(t)}$. Let $\pi_I(\cdot;\theta_I)$ be the solution actor NN of the ID agent with parameter θ_I. Then, the ID solution action $x_j^{(t)}$ in (19), which equals the offloading ratio $\lambda_j^{(t)}$, is expressed as

$$x_j^{(t)} = \pi_I(\mathbf{w}_j^{(t)};\theta_I). \tag{32}$$

The latency constraint in (13) always becomes feasible if $x_j^{(t)}$ lies within a bounded range $[0, \lambda_{\max,j}^{(t)}]$, where an upperbound $\lambda_{\max,j}^{(t)}$ is given by

$$\lambda_{\max,j}^{(t)} = \min\left\{1, \frac{\tau T}{I_j}/(\frac{1}{R_{u,j}^{(t)}} + \frac{\delta}{R_{d,j}^{(t)}} + \frac{C}{f_j^{(t)}})\right\}. \tag{33}$$

To guarantee $x_j^{(t)} \in [0, \lambda_{\max,j}^{(t)}]$, the output activation of $\pi_I(\cdot;\theta_I)$ is set to the sigmoid function multiplied by $\lambda_{\max,j}^{(t)}$. To calculate $\lambda_{\max,j}^{(t)}$, ID agent j needs to know its CPU frequency allocation $f_j^{(t)}$, which is, in fact, determined at the UAV agent. To this end, the IDs are assumed to be equipped with a copy of the UAV actor NN $\gamma_F(\cdot;\zeta_F)$ in (30). This can be achieved by an offline training procedure which optimizes all actor NNs jointly before its real-time inference step, as will be discussed in Section VI. Along with the UAV message $m_0^{(t)}$ and the normalization operation in (31), each ID agent readily gets its CPU frequency $f_j^{(t)}$ without incurring additional communication with the UAV and other ID agents.

Algorithm 1 Cooperative inference of the proposed C-MADDPG

1. *Uplink coordination*
   ID agent j ($\forall j \in \mathcal{N}$) creates $m_j^{(t)}$ from (22) and sends it to the UAV agent.
2. *Downlink coordination*
   The UAV agent computes $m_0^{(t)}$ from (23)-(27) and multicasts it to all ID agents.
3. *Decision at UAV agent*
   The UAV agent calculates the solution action $x_0^{(t)}$ from (28)-(30).
4. *Decision at ID agents*
   Each ID agent j ($\forall j \in \mathcal{N}$) obtains the solution action $x_j^{(t)}$ from (32) and (33).

Algorithm 1 summarizes the cooperative inference among the UAV and ID agents of the proposed C-MADDPG framework. At the beginning of the time slot, ID agent j individually generates the ID message $m_j^{(t)}$ using its message actor NN $\mu_I(\cdot;\varphi_I)$ in (22). The resulting messages are conveyed to the UAV agent through the uplink coordination channel. Next, the UAV agent generates the UAV message $\mathbf{m}_0^{(t)}$ based on its message actor NN $\mu_U(\cdot;\varphi_U)$ and multicasts the output message to the ID agents. After this uplink-downlink agent interaction, each agent individually decides its solution action by leveraging the dedicated solution actor NNs $\pi_U(\cdot;\theta_U)$ and $\pi_I(\cdot;\theta_I)$.

It is important to note that the proposed cooperative inference can be realized in a decentralized manner. Both the message and solution actions can be determined at individual agents based only on their local information, e.g., observation $o_j^{(t)}$ and messages $m_j^{(t)}$. As a consequence, neither central information collection steps nor central computing units are necessary to implement Algorithm 1. So far, we have designed cooperative actor NNs and their decentralized execution mechanisms for establishing the proposed MADDPG framework. In the following sections, we discuss the MADDPG training algorithm for the proposed actor NNs.

VI. Joint Training Strategy

We present a joint training policy which optimizes the actor NNs in an end-to-end manner. The CTDE strategy is adopted which trains all NNs centrally in an offline manner and dispatches the trained actor NNs to intended nodes for online and decentralized decisions. This approach guarantees that the identical actor NNs are deployed across all IDs in the training step. To train the actor NNs, we employ a critic NN $Q(s^{(t)}, a^{(t)};\phi)$ with parameter ϕ which estimates the Q-value of the state-action pair $(s^{(t)}, a^{(t)})$, where $a^{(t)}$ stands for the global action collecting all actions of the UAV and ID agents as

$$a^{(t)} = \{a_j^{(t)} : \forall j \in \tilde{\mathcal{N}}\}. \tag{34}$$

The training step leverages a relay buffer $\mathcal{M}$ given as

$$\mathcal{M} = \{(s^{(t)}, a^{(t)}, r^{(t)}, s^{(t+1)}) : \forall t\}. \tag{35}$$

To improve the scalability to the ID population N, transition samples in (35) are generated over random N uniformly distributed within $[N_{\min}, N_{\max}]$. This enables a versatile computation architecture for the critic NN $Q(\cdot;\phi)$ to handle variable-length state and action inputs, which can be realized by a simple masking operation to the input. More precisely, for each transition sample, we uniformly set the ID population $N \in [N_{\min}, N_{\max}]$ and sample the index set of ID agents randomly, i.e., $\mathcal{N} \subset \mathcal{N}_{\max} \triangleq \{1, \cdots, N_{\max}\}$ with $|\mathcal{N}| = N$. The observations $o_j^{(t)}$ for inactive ID agents $j \in \mathcal{N}_{\max} \backslash \mathcal{N}$ are fixed as zero vectors. Then, the state $s^{(t)}$ in (17) concatenates all observations $o_j^{(t)}$ ($\forall j \in \{0\} \bigcup \mathcal{N}_{\max}$) thereby resulting in a masked vector. A similar operation is applied to construct the global action $a^{(t)}$ in (34). As a consequence, the critic NN $Q(s^{(t)}, a^{(t)};\phi)$ designed to process the maximum ID population $N_{\max}$ preserves the scalability to an arbitrary N.

At each training iteration, we randomly sample a mini-batch set $\mathcal{B} \subset \mathcal{M}$ from the replay buffer $\mathcal{M}$. Let $b = (s, a, r, s') \in \mathcal{B}$ be a particular batch sample where s' represents the one-step forward state originated from the current state-action pair (s, a). Also, we define $\psi \triangleq \{\varphi_U, \theta_U, \varphi_I, \theta_I\}$ as the parameter set of all actor NNs. The actor NNs are optimized to maximize the policy objective function $J(\psi)$ which measures the expected Q-value over the mini-batch samples $b \in \mathcal{B}$ as

$$J(\psi) = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} Q(s, A(s; \psi); \phi), \tag{36}$$

where $A(\cdot; \psi)$ indicates a group of actor NNs that take the global action a from the current state s. As a result, the mini-batch stochastic gradient descent (SGD) update strategy of the actor NNs becomes

$$\psi \leftarrow \psi + \eta_A \nabla_\psi J(\psi), \tag{37}$$

where $\eta_A > 0$ denotes the learning rate and ∇_z equals the gradient operator with respect to the variable z.

The critic NN $Q(s, a; \phi)$ is obtained to yield a correct Q-value for a given state-action pair (s, a). The corresponding critic loss function $L(\phi)$ is written by

$$L(\phi) = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \big(y - Q(s, a; \phi)\big)^2, \tag{38}$$

where y stands for the target of the estimated Q-value $Q(s, a; \phi)$ as

$$y = r + \alpha Q(s', A(s'; \psi'); \phi'), \tag{39}$$

with $\alpha \in (0, 1]$. Then, the critic NN is updated as

$$\phi \leftarrow \phi - \eta_C \nabla_\phi L(\phi) \tag{40}$$

where η_C equals the learning rate for the critic NN. In addition, we adopt the soft update strategy for the target actor NN $A(\cdot; \psi')$ and target critic NN $Q(\cdot; \phi')$ expressed by

$$\psi' \leftarrow \kappa_A \psi + (1 - \kappa_A)\psi' \tag{41}$$

$$\phi' \leftarrow \kappa_C \phi + (1 - \kappa_C)\phi', \tag{42}$$

where κ_C and κ_A are the target update rates.

Algorithm 2 presents the joint training process of the proposed C-MADDPG framework. Unlike the existing MADDPG training algorithm, the proposed training procedure involves scalable learning strategies to enhance the generalization ability of the actor and critic NNs. In the initialization phase, the number of IDs N is generated uniformly within $[N_{\min}, N_{\max}]$. Then, we randomly sample the set of active IDs as $\mathcal{N} \subset \mathcal{N}_{\max} \triangleq \{1, \cdots, N_{\max}\}$ with $|\mathcal{N}| = N$. A masking operation is employed for inactive IDs where the local observations $o_j^{(t)}$ for $j \in \mathcal{N}_{\max} \backslash \mathcal{N}$ are replaced with zero vectors. Consequently, the state $s^{(t)}$ contains the observations of the UAV and active IDs. This masking operation is similarly applied to the action $a^{(t)}$ and this can be viewed as an extension of the dropout operation [45] developed for the generalization ability. We discard the local observations and local actions of randomly selected inactive IDs. By doing so, ensemble training of the actor and critic NNs is achieved by producing a number of randomized MEC configurations in the training. This is also beneficial for the actor NNs to learn a generic decentralized decision policy with any given N. The hyperparameters are utilized to improve the exploration capability of the actor NNs by adding the Gaussian noise as

Algorithm 2 Joint training strategy of the proposed C-MADDPG

Initialize ψ, ϕ, ψ', ϕ' σ^2 and ρ.
for episode $e = 1, \cdots, E$
  Initialize the number of IDs $N \in [N_{\min}, N_{\max}]$.
  Sample the set of active IDs $\mathcal{N} \subset \mathcal{N}_{\max}$.
  Generate an initial state $s^{(1)}$ by masking local observations of inactive IDs $\mathcal{N}_{\max} \backslash \mathcal{N}$.
  Update the exploration noise variance as $\sigma^2 \leftarrow \rho\sigma^2$.
  for time slot $t = 1, \cdots, T$
    Calculate $a^{(t)} = A(s^{(t)}; \psi)$ from Algorithm 1.
    Add the exploration noise as $a^{(t)} \leftarrow a^{(t)} + \mathcal{N}(0, \sigma^2)$.
    Mask local actions of inactive IDs in $a^{(t)}$.
    Obtain reward $r^{(t)}$ and the next state $s^{(t+1)}$.
    Store $(s^{(t)}, a^{(t)}, r^{(t)}, s^{(t+1)})$ in the replay buffer $\mathcal{M}$
    Sample the mini-batch set $\mathcal{B} \subset \mathcal{M}$.
    Update the actor NN ψ and the critic NN ϕ from (37) and (40).
    Update the target actor NN ψ' and the target critic NN ϕ' from (41) and (42).
  end
end

$$a^{(t)} = A(s^{(t)}; \psi) + n, \tag{43}$$

where $n \backsim \mathcal{N}(0, \sigma^2)$ stands for the Gaussian random variable. The exploration noise helps the actor NN $A(\cdot; \psi)$ choose new actions that have not been experienced in the training, thereby improving the generalization capability. At the beginning of each episode, we set N uniformly over $[N_{\min}, N_{\max}]$. An initial state s is randomly generated according to predefined UAV and ID deployment scenarios. Also, we decrease the exploration noise variance σ^2 by the decaying ratio ρ. Thus, the power of the exploration noise in (43) is gradually reduced as the training continues.

Each episode consists of T time slots. At each time slot, we execute Algorithm 1 to yield the action $a^{(t)}$ for the current state $s^{(t)}$. Then, the exploration noise is injected into the action $a^{(t)}$ as in (43). The interaction with the MEC network produces the reward $r^{(t)}$ for the state-action pair $(s^{(t)}, a^{(t)})$ as well as the next state $s^{(t+1)}$. The resulting transition sample $(s^{(t)}, a^{(t)}, r^{(t)}, s^{(t+1)})$ is stored to the replay buffer $\mathcal{M}$. Next, we randomly sample the mini-batch set $\mathcal{B}$ from the replay buffer $\mathcal{M}$ uniformly, which is utillized for the training of the actor NNs and critic NN based on their update rules in (37) and (40). It is then followed by the modification steps of the target NNs in (41) and (42). These procedures are repeated for T time slots, which complete one training episode. The entire training step elapses total E episodes.

The proposed joint training algorithm is conducted centrally in an offline manner. This is due to the critic NN $Q(s, a; \phi)$ which estimates the Q-value of the global state-action pair

TABLE I: Simulation Parameters

Symbol	Settings	Symbol	Settings
τ, T	0.2 s, 10	$\chi_{\text{LoS}}, \chi_{\text{NLoS}}$	3 dB, 23 dB
I_j, δ	$[2, 20]$ Gbits, 0.2	K_1, K_2	$11.95, 0.14$
C, ϑ	$1550, 10^{-28}$	B, N_0	10 MHz, -130 dBm
ρ_0, α	-38 dB, 2	$\alpha_{\text{C}}, \alpha_{\text{A}}$	$1 \times 10^{-3},\ 1 \times 10^{-4}$
p_U, p_D	1 W, 10 W	$v_{\max},\ f_{\max}$	50 m/s, 40 GHz
$\rho,\ \sigma^2$	$0.45, 0.9995$		

(s, a), thereby incurring the centralized information collection process. A network cloud can be employed to train the actor NNs and critic NN jointly. By doing so, the SGD updates in (37) and (40) can be realized with the aid of the shared reward function (20). Notice that the critic NN is employed only in the training phase for the optimization of the actor NNs. Therefore, once the training is completed, the critic NN can be discarded, and only the optimized actor NNs are dispatched to their desired agents for real-time task offloading and positioning decisions.

Since the proposed C-MADDPG framework facilitates individual computation architectures of the actor NNs, the decentralized execution of the UAV and ID agents is guaranteed based on Algorithm 1. In the decentralized inference step, the agents share the coordination messages generated from the trained message actor NNs, whereas no parameter exchanges are required for the parameter sharing policy. In fact, this can be easily ensured in the training step since they are optimized under the identical GD update rule (37). This offline training process incurs no additional communication overheads in the online inference step.

VII. Numerical Results

We present numerical results validating the proposed C-MADDPG. Unless otherwise stated, simulation parameters are fixed as in Table I. The critic NN has four fully-connected hidden layers each with 512, 256, 128, and 64 neurons. The message actor NNs $\mu_U(\cdot;\varphi_U)$ and $\mu_I(\cdot;\varphi_I)$ are built with three layers each having 128 neurons. Also, we leverage four fully-connected layers with 128 neurons for constructing the solution actor NNs $\pi_U(\cdot;\theta_U)$ and $\pi_I(\cdot;\theta_I)$. Output layers of the message actor NNs adopt the ReLU activation functions, whereas those of the solution actor NNs are set to the hyperbolic tangent function. Total $E = 10^5$ episodes and $T = 10$ time slots are considered in the training where each time slot consists of $|\mathcal{B}| = 256$ mini-batch samples. For the initialization, the UAV and IDs are uniformly distributed in a 100 m-by-100 m square area, and the altitude of the UAV is restricted to a bounded range $[0\text{ m}, 60\text{ m}]$. The trained C-MADDPG is tested with 10^4 episodes.

We consider the following benchmark schemes.

- *Vanilla MADDPG* [36]: Message exchanges among the agents are not allowed. Thus, each agent is equipped with the solution actor NN only and produces $x_j^{(t)}$ based on the partial observation $o_j^{(t)}$.
- *Single agent DDPG (SADDPG)*: An ideal centralized DRL scheme is adopted where a super actor NN decides the solution actions of all the UAV and ID agents jointly based on the state input $s^{(t)}$.

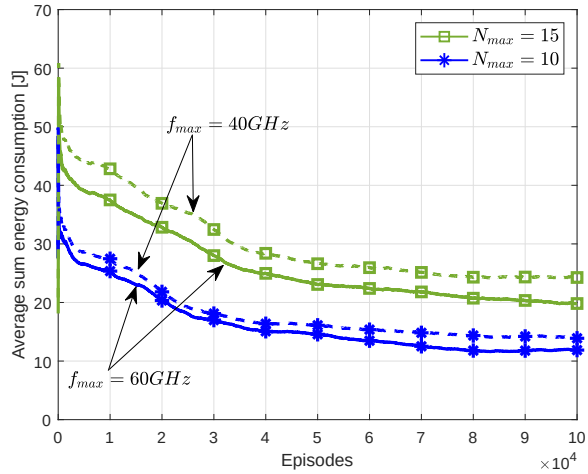

Fig. 3: Convergence of the training process for different $N_{\max}$

- *C-MADDPG with GraphSage (C-MADDPG-GS)*: Instead of the GAT, the actor NN of the UAV agent is implemented with the GraphSage [46].
- *Naive*: The UAV position is simply determined as a centroid of the IDs. Then, all IDs offload their tasks to the UAV, and the computing resources are equally allocated.

The vanilla MADDPG is a special case of the proposed C-MADDPG with no messages exchanged among agents. The SADDPG assumes an ideal centralized system where the UAV and IDs can share their observations perfectly. Unlike the proposed C-MADDPG, a single actor NN architecture of the SADDPG fails to achieve the scalability for varying N. Therefore, the SADDPG needs to be trained at each given N. For this reason, the SADDPG baseline provides unachievable upperbound performance of the proposed C-MADDPG approach. In the C-MADDPG-GS, the intermediate vector $\mathbf{w}_j^{(t)}$ of agent j in (26) is computed as the concatenation of the corresponding feature vector $\mathbf{e}_j^{(t)}$ in (24) and the aggregation of others $\mathbf{e}_k^{(t)} (\forall k \in \tilde{\mathcal{N}} \backslash \{j\})$ as

$$\mathbf{w}_j^{(t)} = \left\{ \mathbf{e}_j^{(t)}, \sum_{k \in \tilde{\mathcal{N}} \backslash \{j\}} \mathbf{e}_k^{(t)} \right\}. \tag{44}$$

In the C-MADDPG-GS, we halve the dimension of the feature vector $\mathbf{e}_j^{(t)}$ so that the concatenation $\mathbf{w}_j^{(t)}$ occupies the identical RBs with the proposed C-MADDPG.

Fig. 3 depicts the convergence behavior of the training process of the proposed C-MADDPG method in terms of the total energy consumption performance. We plot the moving average energy consumption over 10^4 episodes. From the figure, we can check that the proposed C-MADDPG converges within 10^5 episodes for all simulated $N_{\max}$. This implies the effectiveness of the proposed training strategy for handling a number of IDs. Since a small $N_{\max}$ results in simple training processes, it is beneficial to set $N_{\max} = 10$.

Fig. 4 compares the energy consumption performance of various schemes by changing N. We set the ID population regime in the training to $N \in [5, 10]$, and the test performance

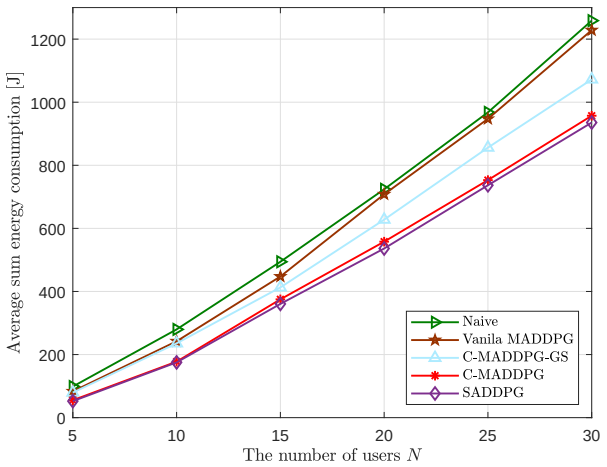

Fig. 4: Average sum energy consumption with respect to N

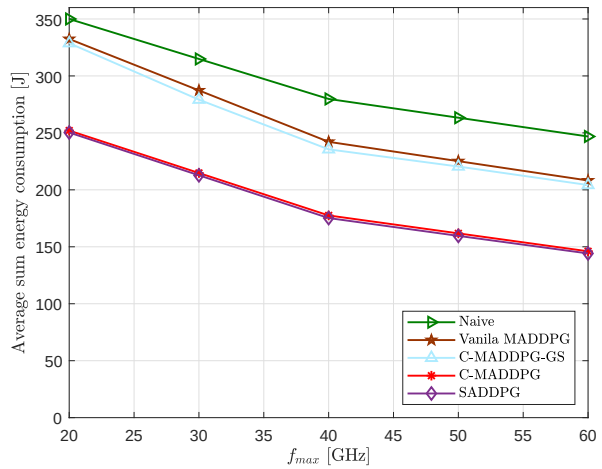

Fig. 5: Average sum energy consumption with respect to f_{max}

of the trained C-MADDPG is evaluated over $N \in [5, 30]$. Thus, the actor NNs cannot observe training samples with $N \geq 10$. Nevertheless, the proposed C-MADDPG shows negligible loss to the upperbound performance generated by the ideal centralized SADDPG method which is trained at each given N. This proves the scalability of the proposed approach where the actor NNs optimized at a small N can be readily applied to larger networks. The gap between the vanilla MADDPG and the naive benchmark scheme decreases as N grows. Without a proper agent interaction mechanism, the solution actor NNs of the vanilla MADDPG cannot provide efficient MEC management solutions and they simply converge to a suboptimal policy of the naive baseline. Based on this result, we can conclude that the message actor NNs play crucial roles in controlling the UAV and IDs in a decentralized manner. Also, the performance of the C-MADDPG-GS is degraded compared to the ideal SADDPG method. A simple sum pooling operation of the GS method fails to capture the importance of each ID agent in generating the downlink coordination message at the UAV agent. For this reason, the C-MADDPG-GS presents a large performance gap to the C-MADDPG. Thus, it is concluded that the GAT-based UAV message actor design is essential to control separate ID agents in a decentralized manner.

Fig. 5 exhibits the energy consumption performance of various schemes by changing the maximum CPU frequency $f_{\max}$ with $N = 10$. The UAV-aided MEC network can save the operating energy of the IDs as $f_{\max}$ increases. As expected, the proposed C-MADDPG outperforms other baseline schemes and provides almost identical performance to the upperbound SADDPG method. This validates the effectiveness of the C-MADDPG for optimizing appropriate MEC management solutions regardless of $f_{\max}$.

The adaptability to time-varying ID population N is investigated in Fig. 6 which depicts the energy consumption as a function of the time slot. The number of IDs is assumed to change at every 10 time slots. We set $N = 5$ for the first 10 time slots and then changes to $N = 15$ for the next 10 time slots. For the last 10 time slots, the ID population is

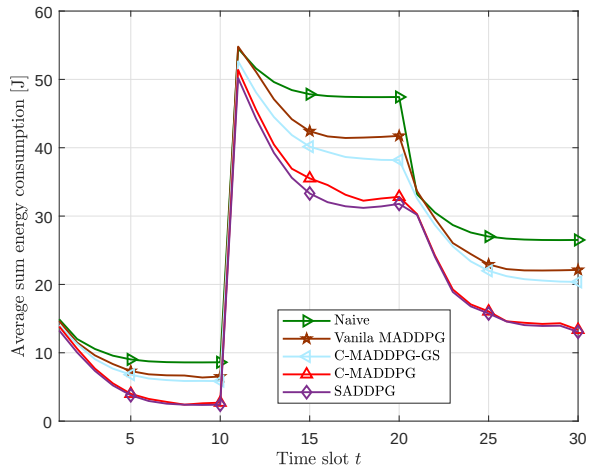

Fig. 6: Average sum energy consumption with varying N

fixed as $N = 10$. The energy consumption of all schemes highly fluctuates at the transition time slots and then is gradually reduced as the actor NNs yield convergent policies. Since the vanilla MADDPG relies on long-term collaboration through rewards rather than real-time cooperation based on observations, it adapts slowly to environment changes. In contrast, the proposed C-MADDPG shows fast convergence by means of online information exchange among agents. Thanks to the scalability, the proposed C-MADDPG can handle such highly fluctuating MEC configurations only with a sole training process. On the contrary, the non-scalable baselines such as the SADDPG and vanilla MADDPG resort to several trained actor NNs dedicated to each possible N. Also, the proposed approach with the GAT architecture achieves the performance of the ideal SADDPG method, proving the effectiveness of the proposed scheme.

Table II compares the inference time complexity of trained actor NNs by evaluating the average CPU running time for executing 10^4 episodes. The computation complexity of the proposed C-MADDPG remains unchanged with ID

TABLE II: Average CPU running time [msec]

N	10	15	20	25	30
SADDPG	2.30	2.33	2.35	2.37	2.39
C-MADDPG	2.19	2.19	2.19	2.19	2.19

populations due to its decentralized and parallel architecture. For this reason, the inference complexity of the proposed C-MADDPG is slightly lower than that of the centralized SADDPG without incurring performance degradation.

VIII. Conclusion

This work has proposed a novel C-MADDPG approach for the decentralized control of the UAV-aided MEC networks where the UAV and IDs can only get access to their local observations only. To build a valid decentralized decision-making policy, it is necessary to develop a proper coordination protocol, in particular, interaction messages bearing sufficient statistics of the optimal solutions of others. The considered problem has been formalized into a cooperative multi-agent POMDP which includes interaction messages as well as solutions of individual agents as action variables. Such dual actions request two different actor NNs, the message actor NN and solution actor NN, each of which accounts for the agent coordination and solution optimization. For effective message aggregation, the message actor NN at the UAV adopts the GAT architecture. Along with the parameter sharing policy, this graph-inspired structure leads to versatile operations that do not depend on the network size. Also, the joint training algorithm of all actor NNs has been proposed. To achieve the scalability to the ID population, the proposed C-MADDPG has been optimized over arbitrary random number of the ID agents. Numerical results have demonstrated the superiority of the proposed scheme over existing schemes.

References

[1] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.

[2] C. L. Chen, C. G. Brinton, and V. Aggarwal, "Latency minimization for mobile edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2233–2247, Apr. 2023.

[3] L. Zhang, Y. Sun, Z. Chen, and S. Roy, "Communications-caching-computing resource allocation for bidirectional data computation in mobile edge networks," *IEEE Trans. Commun.*, vol. 69, no. 3, pp. 1496–1509, Nov. 2021.

[4] M. Masoudi and C. Cavdar, "Device vs edge computing for mobile services: Delay-aware decision making to minimize power consumption," *IEEE Trans. Mobile Comput.*, vol. 20, no. 12, pp. 3324–3337, Dec. 2021.

[5] K. Zhang, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Mobile edge computing and networking for green and low-latency internet of things," *IEEE Commun. Mag.*, vol. 56, no. 5, p. 39–45, May 2018.

[6] M. Kim, H. Lee, S. Hwang, M.Kim, M. Debbah, and I. Lee, "Decentralized learning framework for hierarchical wireless networks: A tree neural network approach," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17 780–17 796, May 2024.

[7] J. Park, S. Solanki, S. Baek, and I. Lee, "Latency minimization for wireless powered mobile edge computing networks with nonlinear rectifiers," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8320–8324, Aug. 2021.

[8] M. Wu, W. Qi, J. Park, P. Lin, L. Guo, and I. Lee, "Residual energy maximization for wireless powered mobile edge computing systems with mixed-offloading," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 4523–4528, Apr. 2022.

[9] N. Kiran and C. Pan and S. Wang and C. Yin, "Joint resource allocation and computation offloading in mobile edge computing for SDN based wireless networks," *J. Commun. Netw.*, vol. 22, no. 1, pp. 1–11, Feb 2020.

[10] Y. Zeng, R. Zhang, and T. J. Lim, "Wireless communications with unmanned aerial vehicles: opportunities and challenges," *IEEE Commun. Mag.*, vol. 54, no. 5, pp. 36–42, May 2016.

[11] N. H. Motlagh, T. Taleb, and O. Arouk, "Low-altitude unmanned aerial vehicles-based internet of things services: Comprehensive survey and future perspectives," *IEEE Internet Things J.*, vol. 3, no. 6, pp. 899–922, Dec. 2016.

[12] B. Li, Z. Fei, and Y. Zhang, "UAV communications for 5G and beyond: Recent advances and future trends," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2241–2263, Apr. 2019.

[13] Z. Liu, J. Qi, Y. Shen, K. Ma, and X. Guan, "Maximizing energy efficiency in UAV-assisted NOMA–MEC networks," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22 208–22 222, Dec 2023.

[14] H. Zhou, Z. Wang, G. Min, and H. Zhang, "UAV-Aided computation offloading in mobile-edge computing networks: A stackelberg game approach," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6622–6633, Apr 2023.

[15] H. Xie, T. Zhang, X. Xu, D. Yang, and Y. Liu, "Joint sensing, communication and computation in UAV-assisted systems," *IEEE Internet Things J.*, Mar 2024, to be published.

[16] Y. Liu, S. Xie, and Y. Zhang, "Cooperative offloading and resource management for UAV-enabled mobile edge computing in power IoT system," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 12 229–12 239, Oct. 2020.

[17] L. Zhang *et al.*, "Task offloading and trajectory control for UAV-assisted mobile edge computing using deep reinforcement learning," *IEEE Access*, vol. 9, pp. 53 708–53 719, Apr. 2021.

[18] Q. Liu, L. Shi, L. Sun, J. Li, M. Ding, and F. Shu, "Path planning for UAV-mounted mobile edge computing with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 5, pp. 5723–5728, May 2020.

[19] L. Wang, P. Huang, K. Wang, G. Zhang, L. Zhang, N. Aslam, and K. Yang, "Rl-based user association and resource allocation for multi-UAV enabled MEC," *in Proc. Int. Wireless. Commun. Mobile Comput. Conf.*, pp. 741–746, Jun. 2019.

[20] H. Wang, H. Ke, and W. Sun, "Unmanned-aerial-vehicle-assisted computation offloading for mobile edge computing based on deep reinforcement learning," *IEEE Access*, vol. 8, pp. 180 784–180 798, Oct. 2020.

[21] Y. Peng, Y. Liu, and H. Zhang, "Deep reinforcement learning based path planning for UAV-assisted edge computing networks," *in Proc. IEEE Wireless Commun. Netw. Conf*, pp. 1–6, Mar. 2021.

[22] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.

[23] S. Hwang, J. Park, H. Lee, M. Kim, and I. Lee, "Deep reinforcement learning approach for UAV-assisted mobile edge computing networks," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 3839–3844.

[24] Z. Chen and X. Wang, "Decentralized computation offloading for multi-user mobile edge computing: A deep reinforcement learning approach," *EURASIP J. Wireless. Commun. Netw.*, vol. 2020, no. 188, pp. 1–21, Sep. 2020.

[25] F. Song, H. Xing, X. Wang, S. Luo, P. Dai, Z. Xiao, and B. Zhao, "Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing,," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2023.

[26] Y. M. Park, S. S. Hassan, Y. K. Tun, Z. Han, and C. S. Hong, "Joint trajectory and resource optimization of MEC-assisted UAVs in sub-THz networks: A resources-based multi-agent proximal policy optimization DRL with attention mechanism," *IEEE Trans. Veh. Technol.*, early access, doi: 10.1109/TVT.2023.3311537.

[27] B. Zhang, B. Tang, and F. Xiao, "Robust computation offloading and trajectory optimization for multi-UAV-assisted MEC: A multi-agent DRL approach," *IEEE Internet Things J.*, early access, doi: 10.1109/JIOT.2023.3300718.

[28] B. Li, W. Liu, W. Xie, N. Zhang, and Y. Zhang, "Adaptive digital twin for UAV-assisted integrated sensing, communication, and computation networks," *IEEE Trans. Green. Commun. Netw.*, vol. 7, no. 4, pp. 10 497–10 509, Dec. 2023.

[29] S. Hwang, H. Lee, J. Park, and I. Lee, "Decentralized computation offloading with cooperative UAVs: multi-agent deep reinforcement

learning perspective," *IEEE Wireless Commun.*, vol. 29, no. 4, pp. 24–31, Aug. 2022.

[30] L. Wang *et al.*, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cogn. Commun. New.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.

[31] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent DRL for task offloading and resource allocation in multi-UAV enabled IoT edge network," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4534–4547, Dec. 2021.

[32] A. Gao, Q. Wang, W. Liang, and Z. Ding, "Game combined multi-agent reinforcement learning approach for UAV assisted offloading," *IEEE Trans. Veh. Technol.*, vol. 70, no. 12, pp. 12 888–12 901, Dec. 2021.

[33] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.

[34] N. Zhao, Z. Ye, Y. Pei, Y. C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sept. 2022.

[35] Z. Ji, S. Wu, and C. Jiang, "Cooperative multi-agent deep reinforcement learning for computation offloading in digital twin satellite edge networks," *IEEE J. Sel. Areas Commun.*, early access, doi: 10.1109/JSAC.2023.3313595.

[36] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multiagent actor-critic for mixed cooperative-competitive environments," *in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, pp. 6382–6393, Jun. 2017.

[37] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Deep reinforcement learning approach for UAV-assisted mobile edge computing networks," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2018, pp. 1–13.

[38] S. Batabyal and P. Bhaumik, "Mobility models, traces and impact of mobility on opportunistic routing algorithms: A survey," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 3, pp. 1679–1707, 3rd Quart.,2015.

[39] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.

[40] M. Mozaffari *et al.*, "Unmanned aerial vehicle with underlaid device-to-device communications: Performance and tradeoffs," *IEEE Trans. Wireless Commun.*, vol. 15, no. 6, pp. 3949–3963, Jun. 2016.

[41] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, April 2019.

[42] Y. Wang, M. Sheng, X. Wang, L. Wang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, p. 4268, Oct. 2016.

[43] R. Goldberg, "Survey of virtual machine research," *IEEE Computer Mag.*, vol. 7, no. 6, pp. 34–45, Jun 1974.

[44] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural. Netw*, vol. 20, no. 1, pp. 61–80, Jan. 2009.

[45] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, Jun 2014.

[46] S. Zhang, H. Tong, J. Xu, and R. Maciejewski, "Graph convolutional networks: A comprehensive review," *Comput. Social Netw.*, vol. 6, no. 1, pp. 1–23, Dec. 2019.